

Лабораторна робота №8

Тема:

Реалізація архітектури Documentation as Code та автоматизованої генерації документації

Мета роботи

Метою лабораторної роботи є:

- закріплення підходу **Documentation as Code (DaC)**;
- набуття практичних навичок **організації документації як частини репозиторію**;
- проектування **Single Source of Truth** для програмної документації;
- освоєння принципів **автоматизованої генерації документації**.

Передумови виконання

Для виконання роботи студент повинен:

- мати виконану **Лабораторну роботу №7**;
- розуміти структуру SSD, SDD, ISD;
- володіти базовими навичками роботи з Git.

Завдання лабораторної роботи

Студенту необхідно **реалізувати повноцінну систему Documentation as Code** для проекту, спроектованого в ЛР-7, із генерацією статичного сайту документації.

Порядок виконання роботи

1. Створення репозиторію документації

1. Створити окремий Git-репозиторій **або** окрему директорію /docs у репозиторії проекту.
2. Репозиторій повинен містити:
 - контроль версій;
 - історію змін;
 - зрозумілу структуру директорій.

Приклад структури

docs/

```
|— README.md
|— architecture/
|  |— SSD.md
|  |— SDD.md
|  └— ISD.md
|— quality/
|  |— test-strategy.md
|  └— traceability-matrix.md
|— developer/
|  └— onboarding.md
└— mkdocs.yml
```

2. Реалізація Documentation as Code

Необхідно:

- оформити **всі документи у форматі Markdown**;

- забезпечити:
 - читабельність;
 - єдиний стиль;
 - використання заголовків, списків, таблиць.

Забороняється:

- використання Word/PDF як основного формату;
- зберігання документації поза репозиторієм.

Приклад фрагмента Markdown

2. Нефункціональні вимоги

- Продуктивність: до 300 мс на запит
- Доступність: 99.9%
- Безпека: JWT-аутентифікація

3. Проектування Single Source of Truth

Студент повинен:

1. Визначити, **який документ є джерелом істини** для:
 - вимог;
 - архітектури;
 - тестів.
2. Описати правила оновлення документації.

Приклад

Усі функціональні вимоги описуються виключно в SSD. Test Strategy та Test Plan посилаються на ідентифікатори вимог з SSD.

4. Побудова логічної архітектури документації

Необхідно реалізувати:

- README як **точку входу**;
- навігацію між документами;
- логічні переходи (посилання).

Приклад README

Documentation Overview

- [System Specification](architecture/SSD.md)
- [System Design](architecture/SDD.md)
- [Infrastructure](architecture/ISD.md)
- [Quality Assurance](quality/test-strategy.md)

5. Генерація статичного сайту документації

1. Обрати інструмент генерації документації.
2. Налаштувати конфігураційний файл (YAML).
3. Згенерувати статичний сайт документації.

Приклад конфігурації (фрагмент)

site_name: Project Documentation

nav:

- Home: README.md
- Architecture:
 - SSD: architecture/SSD.md
 - SDD: architecture/SDD.md

6. Версійність та актуалізація документації

Студент повинен:

- запропонувати модель версійності документації;
- описати правила оновлення при зміні вимог або архітектури.

Приклад

Версія документації v1.0 відповідає стабільному релізу системи. При зміні API або вимог створюється нова версія документації.

Вимоги до оформлення та здачі

- Формат: **Git-репозиторій**
- Документація: Markdown
- Наявність згенерованого сайту (локально або demo)
- Чітка структура та навігація

Критерії оцінювання

Критерій	Частка
Архітектура DaC	25%
Single Source of Truth	20%
Якість Markdown-документів	25%
Генерація та навігація сайту	20%
Версійність та узгодженість	10%

Контрольні питання

1. У чому полягає принцип Documentation as Code?
2. Що таке Single Source of Truth?
3. Чому Markdown є базовим форматом для документації?
4. Як забезпечується актуальність документації у DaC-підході?